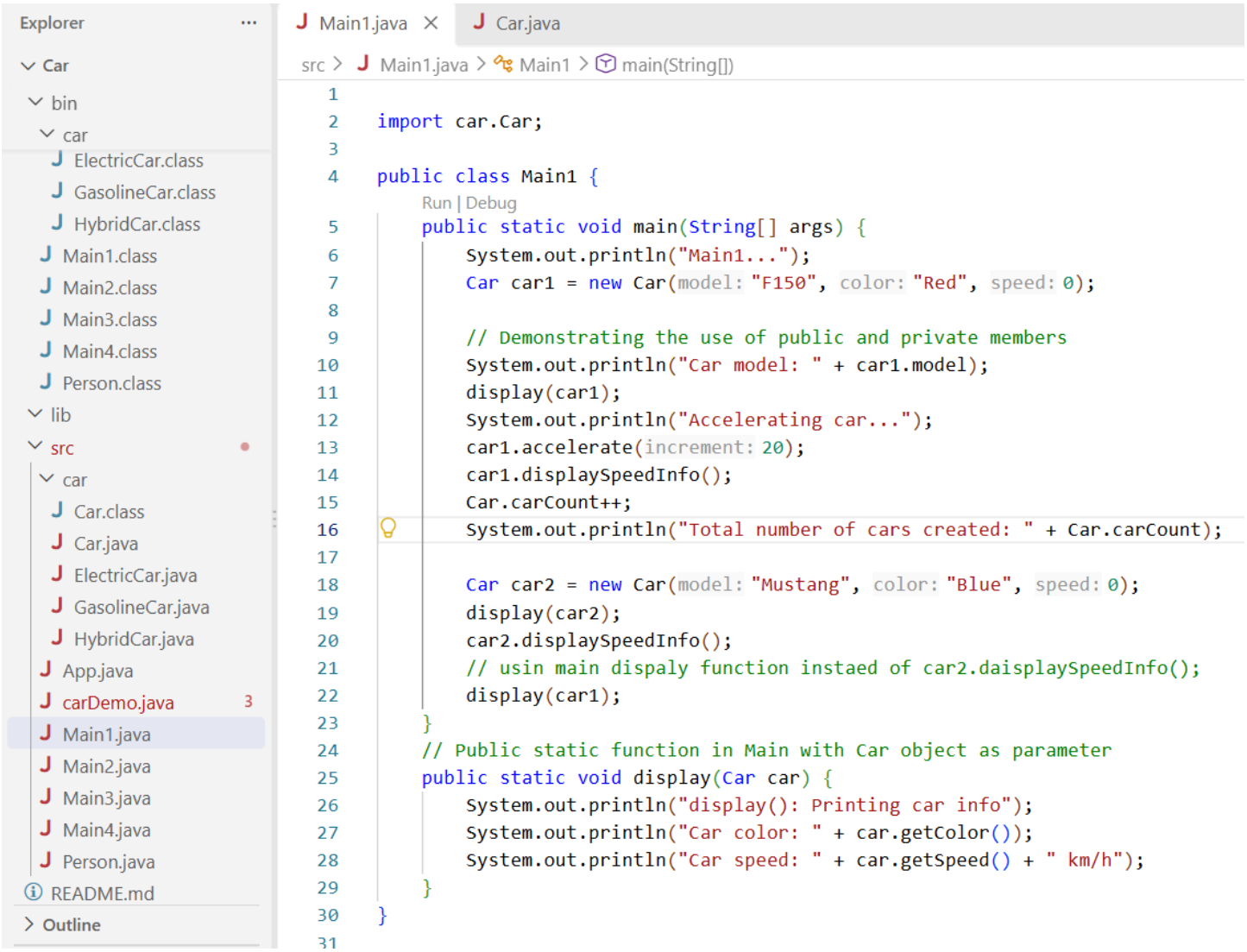
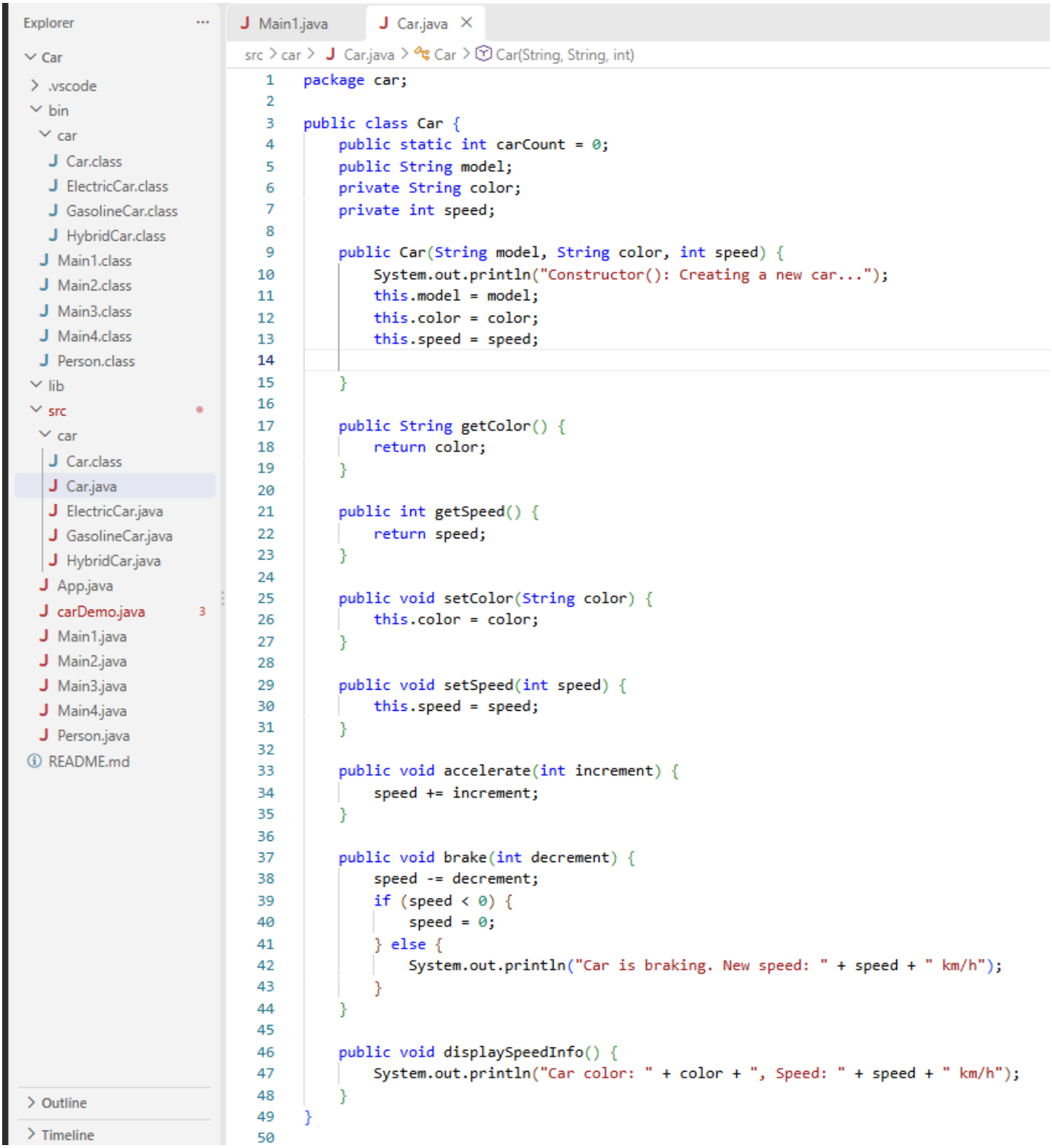
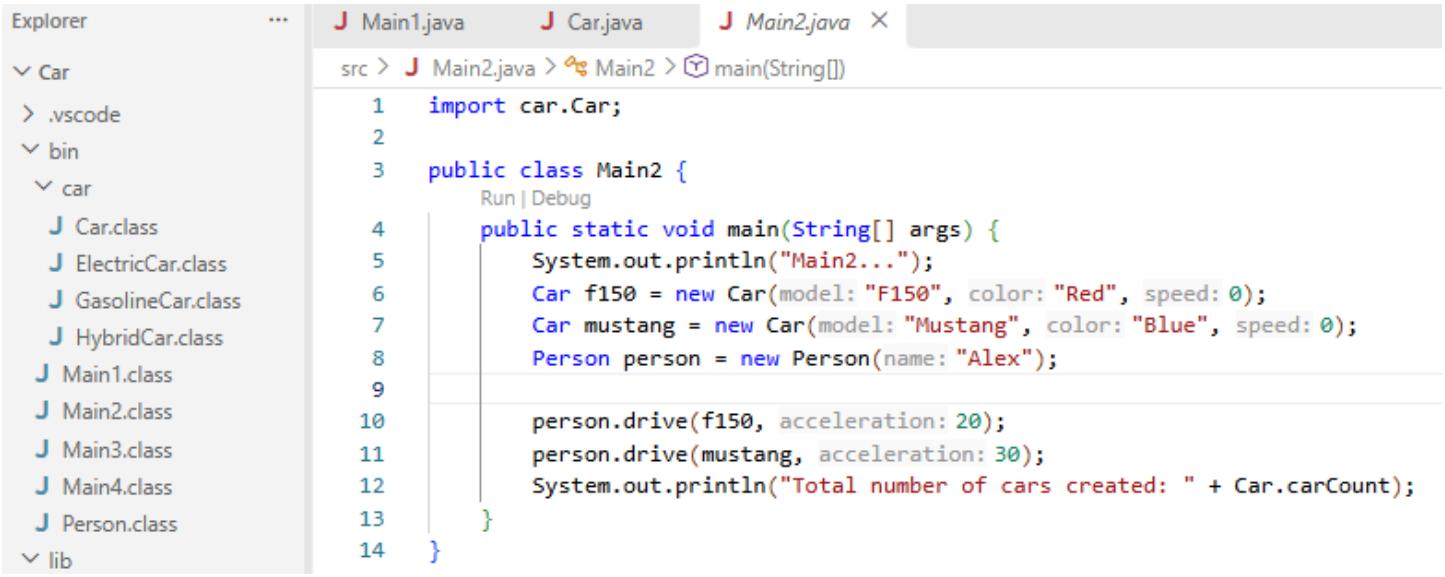
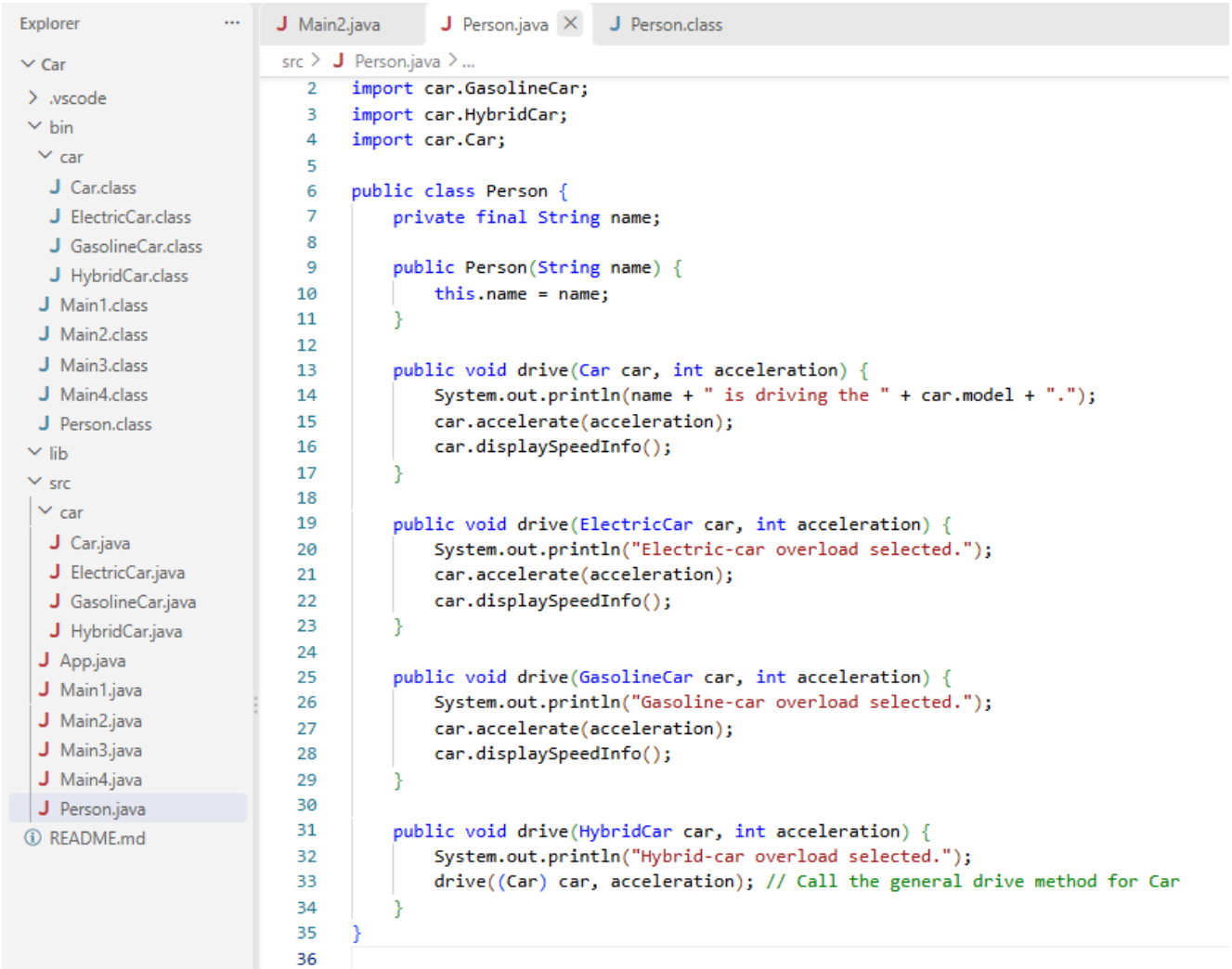
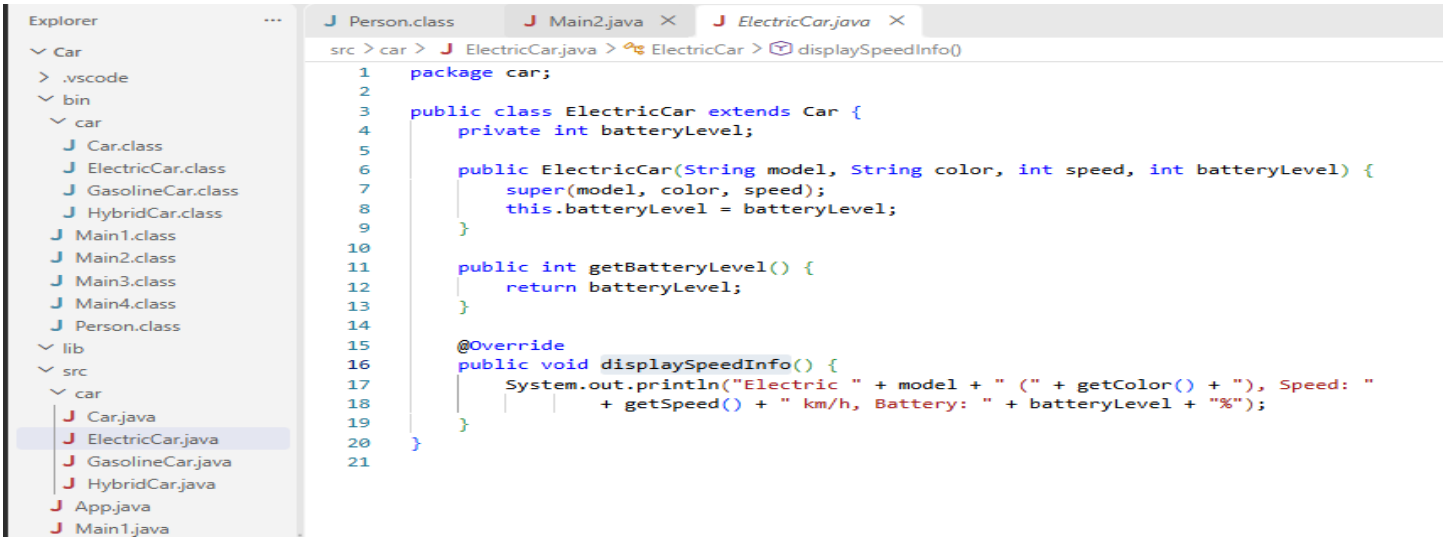

Explorer
Car
bin
car
ElectricCar.class
GasolineCar.class
HybridCar.class
Main1.class
Main2.class
Main3.class
Main4.class
Person.class
lib
src
car
Car.class
Car.java
ElectricCar.java
GasolineCar.java
HybridCar.java
App.java
carDemo.java 3
Main1.java
Main2.java
Main3.java
Main4.java
Person.java
README.md
Outline
Main1.java
Car.java
src > Main1.java > Main1 > main(String[])
1
2 import car.Car;
3
4 public class Main1 {
Run | Debug
5 public static void main(String[] args) {
6 System.out.println("Main1...");
7 Car car1 = new Car(model: "F150", color: "Red", speed: 0);
8
9 // Demonstrating the use of public and private members
10 System.out.println("Car model: " + car1.model);
11 display(car1);
12 System.out.println("Accelerating car...");
13 car1.accelerate(increment: 20);
14 car1.displaySpeedInfo();
15 Car.carCount++;
16 System.out.println("Total number of cars created: " + Car.carCount);
17
18 Car car2 = new Car(model: "Mustang", color: "Blue", speed: 0);
19 display(car2);
20 car2.displaySpeedInfo();
21 // usin main dispaly function instaed of car2.daisplaySpeedInfo();
22 display(car1);
23 }
24 // Public static function in Main with Car object as parameter
25 public static void display(Car car) {
26 System.out.println("display(): Printing car info");
27 System.out.println("Car color: " + car.getColor());
28 System.out.println("Car speed: " + car.getSpeed() + " km/h");
29 }
30 }
31

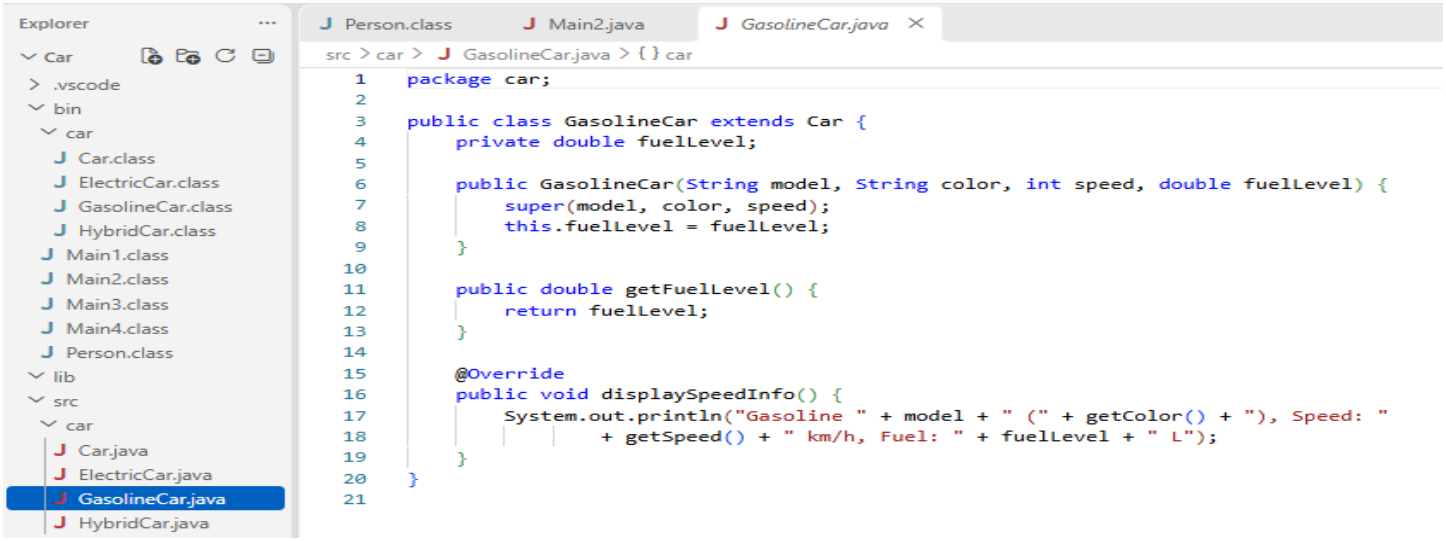

Explorer
Car
.vscode
bin
car
Car.class
ElectricCar.class
GasolineCar.class
HybridCar.class
Main1.class
Main2.class
Main3.class
Main4.class
Person.class
lib
src
car
Car.class
Car.java
ElectricCar.java
GasolineCar.java
HybridCar.java
App.java
carDemo.java 3
Main1.java
Main2.java
Main3.java
Main4.java
Person.java
README.md
Outline
Timeline
Main1.java
Car.java
src > car > Car.java > Car > Car(String, String, int)
1 package car;
2
3 public class Car {
4 public static int carCount = 0;
5 public String model;
6 private String color;
7 private int speed;
8
9 public Car(String model, String color, int speed) {
10 System.out.println("Constructor(): Creating a new car...");
11 this.model = model;
12 this.color = color;
13 this.speed = speed;
14
15 }
16
17 public String getColor() {
18 return color;
19 }
20
21 public int getSpeed() {
22 return speed;
23 }
24
25 public void setColor(String color) {
26 this.color = color;
27 }
28
29 public void setSpeed(int speed) {
30 this.speed = speed;
31 }
32
33 public void accelerate(int increment) {
34 speed += increment;
35 }
36
37 public void brake(int decrement) {
38 speed -= decrement;
39 if (speed < 0) {
40 speed = 0;
41 } else {
42 System.out.println("Car is braking. New speed: " + speed + " km/h");
43 }
44 }
45
46 public void displaySpeedInfo() {
47 System.out.println("Car color: " + color + ", Speed: " + speed + " km/h");
48 }
49 }
50

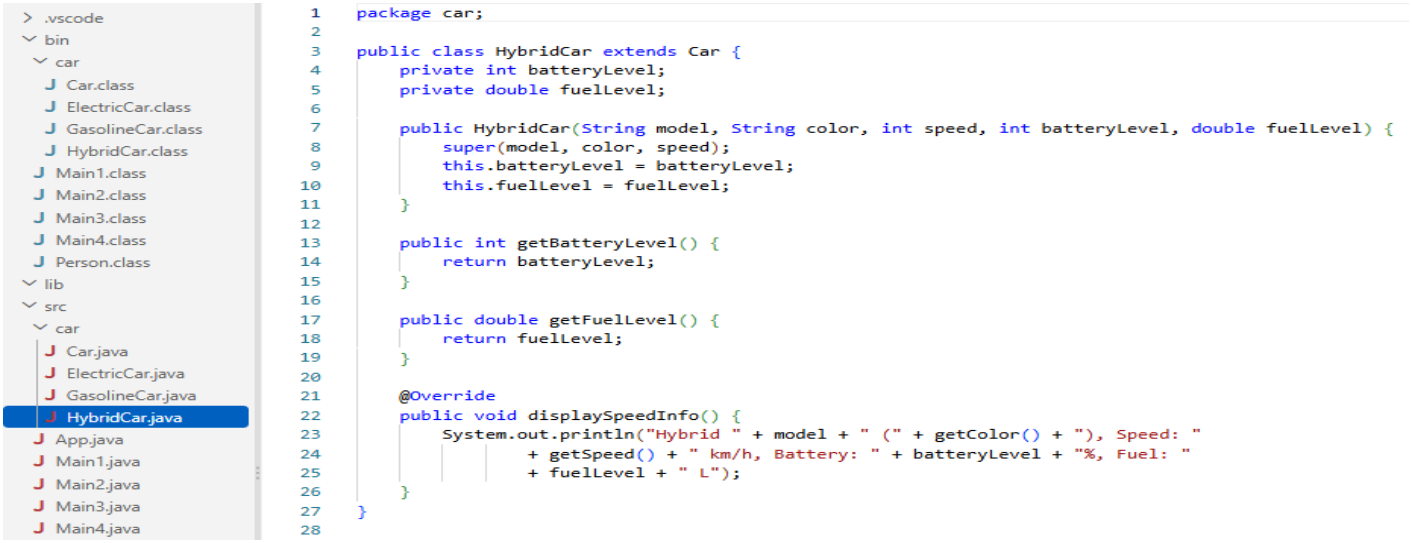

Explorer
Car
.vscode
bin
car
Car.class
ElectricCar.class
GasolineCar.class
HybridCar.class
Main1.class
Main2.class
Main3.class
Main4.class
Person.class
lib
Main1.java
Car.java
Main2.java
src > Main2.java > Main2 > main(String[])
1 import car.Car;
2
3 public class Main2 {
Run | Debug
4 public static void main(String[] args) {
5 System.out.println("Main2...");
6 Car f150 = new Car(model: "F150", color: "Red", speed: 0);
7 Car mustang = new Car(model: "Mustang", color: "Blue", speed: 0);
8 Person person = new Person(name: "Alex");
9
10 person.drive(f150, acceleration: 20);
11 person.drive(mustang, acceleration: 30);
12 System.out.println("Total number of cars created: " + Car.carCount);
13 }
14 }


Explorer
Car
.vscode
bin
car
Car.class
ElectricCar.class
GasolineCar.class
HybridCar.class
Main1.class
Main2.class
Main3.class
Main4.class
Person.class
lib
src
car
Car.java
ElectricCar.java
GasolineCar.java
HybridCar.java
App.java
Main1.java
Main2.java
Main3.java
Main4.java
Person.java
README.md
Main2.java
Person.java
Person.class
src > Person.java > ...
2 import car.GasolineCar;
3 import car.HybridCar;
4 import car.Car;
5
6 public class Person {
7 private final String name;
8
9 public Person(String name) {
10 this.name = name;
11 }
12
13 public void drive(Car car, int acceleration) {
14 System.out.println(name + " is driving the " + car.model + ".");
15 car.accelerate(acceleration);
16 car.displaySpeedInfo();
17 }
18
19 public void drive(ElectricCar car, int acceleration) {
20 System.out.println("Electric-car overload selected.");
21 car.accelerate(acceleration);
22 car.displaySpeedInfo();
23 }
24
25 public void drive(GasolineCar car, int acceleration) {
26 System.out.println("Gasoline-car overload selected.");
27 car.accelerate(acceleration);
28 car.displaySpeedInfo();
29 }
30
31 public void drive(HybridCar car, int acceleration) {
32 System.out.println("Hybrid-car overload selected.");
33 drive((Car) car, acceleration); // Call the general drive method for Car
34 }
35 }
36

Inheritance concepts









Polymorphism

